

## PIE: study guide

Paper: Learning Performance-Improving Code Edits. Alexander Shypula et al. International Conference on Learning Representations (ICLR), 2024. Source: arXiv:2302.07867v5, 26 April 2024, 25 pages.

Prepared 6 October 2026. Page references identify the downloaded PDF. These notes summarize and critically assess the paper; they do not report a replication. The structure follows Keshav's three-pass approach [2].

### Pass 1: the central idea

LLMs can learn high-level source-code optimizations from examples of slower and faster solutions. PIE contributes optimization pairs, a deterministic simulation-based evaluator, and methods for adapting models through retrieval and fine-tuning. The strongest headline result is obtained by generating multiple candidates and choosing the fastest one that passes all available tests. It is not a one-shot guarantee. [1, pp. 1-6]

### The five Cs

**Category:** Dataset and evaluation infrastructure, plus an empirical comparison of LLM adaptation methods. **Context:** Compilers optimize many low-level details, but changes to algorithms, data structures, and APIs can require source-level reasoning. [1, pp. 1-4]

**Correctness:** A candidate is treated as correct if it passes every benchmark test. The benchmark uses a CPU simulator to reduce timing noise. Test adequacy and simulator representativeness remain separate assumptions. [1, pp. 3-6]

**Contributions:** Performance-improving program pairs; gem5-based runtime labels; dynamic retrieval, performance-conditioned fine-tuning, and self-play data augmentation. [1, pp. 3-5]

**Clarity - reviewer assessment:** The method and tables are accessible, but Best@k, fallback-to-original scoring, correctness, and speedup must be understood together.

### Scope and vocabulary

**This version evaluates C++ competitive-programming solutions.** Do not describe its 77,967 training pairs or headline results as Python experiments. Earlier PIE material and subsequent adaptations need separate version checks. [1, p. 3]

**Slow-fast pair:** Two accepted submissions for the same task with an observed improvement. **gem5:** A simulator of CPU behavior. **HQ:** A smaller, curated high-quality training subset. **Best@k:** Select the fastest test-passing candidate from k generations. **Self-play:** Generate extra training problems and optimize their solutions to obtain synthetic pairs. [1, pp. 3-6]

### The important qualification

The reported mean speedup assigns 1.0x to a case when generated code is incorrect or slower, because the original can be retained. This evaluates optimization with validation and fallback. It does not describe the speed or correctness of blindly accepting every output. [1, p. 6]

## Pass 2: data, methods, and evaluation

### Constructing the benchmark

Collect each programmer's accepted CodeNet submissions for the same problem, form chronological pairs, and keep pairs exceeding a 10% runtime reduction under the dataset filter. Relabel runtime with gem5 because reported CodeNet times can disagree for identical programs. Split by programming problem, rather than merely by individual code pair. [1, p. 3]

| Split      | Pairs  | Distinct problems |
|------------|--------|-------------------|
| Training   | 77,967 | 1,474             |
| Validation | 2,544  | 77                |
| Test       | 978    | 41                |

The test set therefore has 978 examples, but only 41 distinct problem identities. Tests combine CodeNet and AlphaCode material; the test split has a median of 104 cases per problem. Slow cases that exceed the simulator budget are excluded. More than 42.8 million simulations are reported for dataset labeling. [1, pp. 3-4, 19]

### Model-adaptation methods

**Prompting:** Compare an instruction, fixed slow-fast examples, and chain-of-thought prompting. **Dynamic retrieval:** Embed source programs with a C++ CodeBERTScore model, retrieve nearby training examples with FAISS, and place their slow-fast pairs in the prompt. Compare one, two, and four retrieved examples. [1, p. 4; Appendix A.7]

**Ordinary fine-tuning:** Train CodeLlama and GPT-3.5 to map slow programs to faster ones. The HQ subset contains 4,085 pairs with high speedups and no more than four pairs per problem. This limits imbalance from frequently occurring problems. [1, pp. 4-5]

**Performance conditioning:** Label faster programs by within-problem performance bins from 1/10 to 10/10 during training. Request 10/10 performance at inference. This encourages stronger optimization but can reduce correctness. [1, p. 5; Appendix A.4]

**Self-play:** Generate synthetic programs; group/filter behaviorally duplicate outputs; optimize them with a model already trained on PIE. Keep large-improvement pairs and limit duplicates. The final augmentation adds 1,485 synthetic pairs. Equivalence here is assessed through available executions, not a proof over every input. [1, pp. 5, 19]

### Measurement and selection

Compile with GCC 9.3.0, C++17, and -O3. Measure with a gem5 Intel Skylake configuration. Generate candidates with temperature 0.7 and report Best@1 and Best@8. %Opt counts correct candidates meeting the paper's optimization threshold; %Correct counts test-passing results. Speedup is old runtime / new runtime, averaged over test examples with the 1.0x fallback described earlier. [1, pp. 4, 6]

## Pass 2: results and supporting analyses

### Main Best@8 comparison

| Method / model                   | % optimized | Mean speedup | % correct |
|----------------------------------|-------------|--------------|-----------|
| Human reference                  | 100.00      | 3.66x        | 100.00    |
| CoT / GPT-3.5                    | 43.05       | 1.60x        | 91.72     |
| Retrieval / GPT-4                | 76.07       | 3.93x        | 95.71     |
| Perf-conditioned / CodeLlama 13B | 66.56       | 5.65x        | 70.96     |
| HQ + self-play / GPT-3.5         | 87.63       | 6.86x        | 95.09     |

Source: [1, p. 6, Table 1; p. 8, Table 4]. The human reference is a set of known improvement pairs, so 100% optimized is a property of the selected dataset. It is not a comparison with humans starting under the same candidate budget. The model rows use test-based candidate selection.

### What the experiments establish

**Retrieval matters:** Relevant examples are substantially stronger than fixed examples or instructions in this setup. For GPT-4, retrieval with four examples gives 3.93x Best@8 mean speedup, versus 1.58x with CoT. [1, p. 7, Tables 2-3]

**Training data quality matters:** CodeLlama 13B trained on all pairs obtains 2.71x Best@8, while HQ training obtains 3.43x. Performance conditioning reaches 5.65x, with lower correctness. [1, p. 8, Table 4]

**Self-play has a modest incremental effect:** GPT-3.5 HQ-only gives 6.74x and 86.71% optimized at Best@8; adding synthetic pairs gives 6.86x and 87.63%. The improvement is larger at Best@1: 2.70x to 3.02x. Do not attribute the entire 6.86x result to self-play. [1, p. 8]

**More candidates change the comparison:** A separate experiment compares 39,129 model generations with 118,841 accepted human submissions, obtaining 9.64x versus 9.56x. This is an observed best-of-many result on the dataset, not a universal upper bound or proof that LLMs optimize better than humans generally. [1, pp. 6, 8]

### Appendix findings worth retaining

Manual analysis of 120 pairs identifies algorithmic edits, I/O changes, data-structure changes, and miscellaneous simplifications. Ten cases improve timing by shrinking allocated arrays; the authors flag that limited tests may fail to expose invalid size assumptions. [1, p. 14, Appendix A.1]

The failure analysis finds compilation errors, many test failures, correct-but-slower outputs, unchanged speed, and improvements below threshold. Mixing performance-conditioning targets improves correctness in an additional experiment. More generated candidates reduce the gap between HQ-only and HQ+self-play. [1, pp. 17-18]

The appendix also gives concrete edits, dataset statistics, synthetic-data generation details, retrieval ablations, training settings, a duplicate-code timing example, LoRA results, and prompt templates. The tested LoRA configurations perform substantially worse than the full fine-tuning configurations; this does not establish that all parameter-efficient training fails. [1, pp. 15-25]

## Pass 3: critical appraisal and research relevance

### What is convincing

The paper tackles two connected problems: learning useful edits and obtaining reproducible performance labels. Problem-disjoint splits, compiler optimization enabled during evaluation, multiple adaptation baselines, and explicit correctness reporting strengthen the evidence. It also openly discusses suspicious speedups and the cost of collecting/training on optimization data. [1, pp. 3-9, 14, 19-22; reviewer assessment]

### Important limitations

**Correctness is test-relative:** Passing tests is not universal semantic equivalence. The array-size examples demonstrate why a fast candidate can exploit missing cases. Further testing should target valid boundary sizes and input distributions, rather than only rerunning existing tests. [1, p. 14; reviewer implication]

**Simulation removes one source of uncertainty:** Deterministic timing does not prove realistic workload coverage or identical behavior on deployed hardware. The paper also notes that a two-minute simulator timeout can introduce variability and affect results. [1, pp. 3-4]

**Repeated selection:** Best@8 requires a selector and adequate tests. Selecting on the same available suite can favor candidates that fit that suite. The study does not establish an error-controlled acceptance decision under arbitrary future workloads. [1, p. 6; reviewer inference]

**Dataset scope:** There are many pairs but relatively few distinct test problems. These are C++ contest programs, not Python services or dependency-rich applications. Program pairs from one problem should not be treated as fully independent tasks in follow-up statistical analysis. [1, pp. 3, 19; reviewer assessment]

**Aggregated speedup:** An arithmetic average of speedup ratios can be influenced by large improvements. Fallback scoring protects the reported performance from slower/incorrect proposals, but only because validation can identify them. Report medians, per-task distributions, rejection rates, and validation cost when adapting the protocol. [1, p. 6; suggested follow-up reporting]

**Historical comparison:** These experiments use older GPT and CodeLlama versions. Their results support the method under the stated configuration, not a ranking of models available today. The paper does not establish freedom from pretraining contamination. [1, pp. 5-6; reviewer scope qualification]

### What it means for our Python study

Already addressed: paired optimization data, test-based rejection, candidate selection, and timing-noise mitigation. These cannot be claimed as new by themselves. The useful question is whether a candidate accepted by such a pipeline remains correct and beneficial when independently validated on unseen valid inputs or after multiple editing rounds.

For a first study, record the original and candidate programs, selection tests, an independent evaluation suite, per-workload effects, and the decision cost. Include identical-program controls to measure false improvements. PIE motivates this: its 500 identical-program timing pairs showed mean apparent speedup of 1.12x, and 1.91x among the top 5%. These are observations from its hardware experiment, not universal noise levels. [1, pp. 3-4; proposed adaptation]

# Recall, reproduction, and source map

## Questions to answer without the paper

1. What do the 77,967 training pairs represent? 2. Why split by problem? 3. Why does the paper use gem5? 4. How do retrieval and performance conditioning differ? 5. What does Best@8 require at evaluation time? 6. How are incorrect or slower generations scored? 7. Why can 6.86x coexist with imperfect correctness? 8. What do the array-size cases reveal about validation?

## Reproduction essentials

Match the released dataset version, problem splits, unit tests, compiler flags, simulator configuration, timeout, model version, generation budget, and candidate-selection rule. Keep HQ and synthetic-data filters explicit. Record generation, simulation, and training costs separately. Full fine-tuning in the paper uses eight 48 GB GPUs; a practical first PhD pilot can evaluate candidate judgments without reproducing that training expense. [1, pp. 3-8, 19-22; proposed practical scope]

## Where to return in the original

| Need                                         | Source pages             |
|----------------------------------------------|--------------------------|
| Motivation and contributions                 | 1-3                      |
| Pairs, splits, tests, measurement noise      | 3-4; Appendix A.5, p. 19 |
| Prompting, retrieval, training, self-play    | 4-5                      |
| Metrics, selection, primary results          | 6-9; Tables 1-4          |
| Edit taxonomy and questionable speedups      | 14                       |
| Worked optimization examples                 | 15-17                    |
| Sampling and error analysis                  | 17-18; Table 5           |
| Training, retrieval ablation, timing example | 19-21                    |
| LoRA and prompt templates                    | 22-25                    |

## References and accessible materials

[1] A. Shypula, A. Madaan, Y. Zeng, U. Alon, J. Gardner, M. Hashemi, G. Neubig, P. Ranganathan, O. Bastani, and A. Yazdanbakhsh. Learning Performance-Improving Code Edits. ICLR, 2024. Source PDF: arXiv v5, 26 April 2024. <https://arxiv.org/pdf/2302.07867v5>

Official ICLR 2024 proceedings record. ICLR is a conference, not a journal.

[2] S. Keshav. How to Read a Paper. ACM SIGCOMM Computer Communication Review 37(3), 83-84, 2007; author revision, 17 February 2016. Reading framework only. <https://svr-sk818-web.cl.cam.ac.uk/keshav/papers/07/paper-reading.pdf>

[3] PIE project site, linked by the paper. Use the artifact corresponding to this published C++ version; do not silently combine it with an earlier Python release. <https://pie4perf.com>

Suggested presentation message: learning from performance-improving edits works substantially better than generic prompting in this benchmark, but successful optimization depends on the evaluator and the tests used to select the result.